

Human-AI Text Checker

Overview

Human-AI Text Checker is a full-stack application that detects whether text is written by an AI model or a human and provides an optional text improvement path to make AI-sounding text more human-like. It uses:

- React frontend for user input, analysis display, and improvement workflow
- Node.js Express API to validate requests, manage history, route inference, and trigger rewrite
- FastAPI inference server with LangChain RAG and Llama API for AI detection and humanization

The system provides probability scores, reasoning, text analysis indicators, and humanized rewrite output.

Architecture

```
React UI (3000)
└─> Node.js API (3001)
    └─> FastAPI Inference (8000)
```

Layers

1. **UI (React)**
 - `ui/src/App.js`, `ui/src/components/CreateAnalysis.js`, `ui/src/components/Analysis.js`
 - Sends analysis request to API and displays AI/human percentages and reasoning.
 2. **API (Node.js Express)**
 - `api/src/index.js`, `api/src/routes/analysis.js`, `api/src/routes/health.js`
 - Validates incoming text requests, proxies to inference, stores analysis history in-memory.
 3. **Inference (FastAPI + LangChain)**
 - `server/main.py`
 - Uses Llama API prompt for AI/human probability, plus optional LangChain Document RAG retrieval from Chroma.
-

Features

- AI vs human probability scoring
 - Reasoning and key indicator outputs
 - RAG-enhanced context retrieval
 - Humanization endpoint to rewrite AI-like text as more human
 - In-memory analysis history API
 - Health and inference system checks
 - Simple responsive React dashboard
-

API Endpoints

Node.js API (default port 3001)

- POST /api/analysis/analyze: analyze text using inference context
- POST /api/analysis/langchain: analyze with LangChain RAG pipeline
- POST /api/analysis/improve: rewrite text to sound more human
- GET /api/analysis/history: list recent analyses
- GET /api/analysis/:id: get stored analysis by ID
- DELETE /api/analysis/:id: delete analysis
- GET /api/health: overall system health
- GET /api/health/inference: inference server health
- GET /api/health/rag: RAG status

FastAPI Inference (default port 8000)

- POST /analyze: main AI/human detection endpoint
 - POST /rewrite: rewrite text to sound more human
 - POST /analyze/langchain: full LangChain RAG endpoint
 - GET /rag/status: RAG collection status
 - GET /rag/test: sample RAG retrieval test
 - GET /health: inference service health
-

Setup and Local Run

1) FastAPI Inference

```
cd server
python -m venv venv
venv\Scripts\activate
```

```
pip install -r requirements.txt
# Configure .env with LLAMA_API_KEY and optionally LLAMA_API_URL, LLAMA_MODEL
uvicorn main:app --reload --port 8000
```

2) Node API

```
cd api
npm install
# Configure .env as needed (INFERENCE_URL default http://localhost:8000)
npm run dev
```

3) React UI

```
cd ui
npm install
npm start
```

Open app at <http://localhost:3000>.

Inference Flow

1. User enters text in frontend
 2. Frontend posts to `/api/analysis/analyze`
 3. API validates text and posts to `http://localhost:8000/analyze`
 4. FastAPI builds prompt and calls Llama API
 5. Inference server returns JSON with probabilities and reasoning
 6. API returns result to frontend and caches in memory
 7. UI displays analysis bars and details
 8. If text is AI-like, user can request rewrite via `/api/analysis/improve`
 9. API calls `http://localhost:8000/rewrite`, returns humanized text
 10. UI displays improved human-like text and allows user review
-

Key Implementation Details

Llama Prompt and Parsing

In `server/main.py`, `LlamaInferenceClient.analyze_text` sends a prompt instructing the model to return valid JSON with:

- ai_probability
- human_probability
- reasoning
- key_indicators

The client parses rich response formats and falls back to regex extraction.

LangChain RAG System

LangChainRAGSystem uses:

- HuggingFaceEmbeddings(all-MiniLM-L6-v2)
- Chroma persistent vector store under server/chroma_db
- RecursiveCharacterTextSplitter and a knowledge base of AI/human patterns
- LLMChain with prompt templates for JSON output

Resilience

- API has rate limiting and security middleware (helmet)
 - API and inference both have health endpoints
 - Inference falls back to neutral prediction when model not configured
-

Extending the Project

Add persistent storage

- Replace in-memory Map in api/src/routes/analysis.js with Postgres/MongoDB.
- Add model persistence for analysis results and user accounts.

Improve analysis UI

- Store analysis history from API to display across reloads.
- Add date sorting, filters, and export features.

Add test coverage

- Add Jest tests for API endpoints.
- Add python unit tests for inference parser logic.

Improve model scoring

- Tune prompt for output reliability.

- Add calibration layer to normalize probabilities.
-

Quick Troubleshooting

- 503 Service Unavailable from API means inference server not running or wrong INFERENCE_URL.
- LLAMA_API_KEY environment variable not set shows in inference logs if key missing.
- CORS errors: ensure frontend uses REACT_APP_API_URL=http://localhost:3001 and API has allowed origin.